

Отчет по разработке SVD рекомендательной системы для платформы Steam

Выполнил: [Шитоев Данил]

11 апреля 2026 г.

Содержание

1	Постановка задачи	2
2	Описание структуры и анализ данных	2
2.1	Структура данных	2
2.2	Статистический анализ после предобработки	2
3	Подход к реализации (FunkSVD)	2
3.1	Математическая модель	3
3.2	Обучение	3
4	Демонстрация на конкретных примерах	3
5	Гиперпараметры модели и влияние их изменения	3
5.1	Сводная таблица результатов	4
5.2	Интерпретация влияния гиперпараметров	4
6	Сравнение с Hybrid KNN	4
7	Выводы	5

1 Постановка задачи

Целью данной работы является разработка и оценка качества рекомендательной системы на основе матричной факторизации (SVD/FunkSVD). В отличие от методов ближайших соседей (KNN), SVD выявляет скрытые (латентные) признаки, объясняющие взаимодействия пользователей с играми, что позволяет лучше обобщать данные и работать с разреженными матрицами.

Ключевые задачи:

1. Реализовать алгоритм FunkSVD с использованием стохастического градиентного спуска (SGD).
2. Провести эксперименты по подбору оптимальной размерности латентного пространства (`n_factors`).
3. Оценить качество рекомендаций с использованием метрик ранжирования (Precision, Recall, HitRate, NDCG) и точности предсказания оценок (RMSE, MAE).
4. Сравнить результаты с предыдущими подходами (Hybrid KNN) и сделать выводы о применимости метода.

2 Описание структуры и анализ данных

Для обучения модели использовался датасет `steam_reco_interactions_clean.csv`.

2.1 Структура данных

- `user_id`: Уникальный идентификатор пользователя.
- `game_id`: Уникальный идентификатор игры.
- `y_recommended`: Бинарная оценка взаимодействия (1 — пользователь рекомендует игру, 0 — не рекомендует). В данной реализации используется как рейтинг.
- `game_name`: Название игры (используется для интерпретации результатов).

2.2 Статистический анализ после предобработки

- **Размер выборки:** 6,931 уникальных пар (пользователь, игра) до разбиения.
- **Разбиение:** Train: 5,544 записей, Test: 1,387 записей.
- **Разреженность:** Данные крайне разрежены. Большинство пользователей имеют очень короткую историю взаимодействий (в среднем < 2 игр на пользователя в агрегированном виде, но в данном сплите объем train достаточен для обучения).
- **Характер оценок:** Бинарные (0/1), что требует осторожности при интерпретации RMSE, так как модель предсказывает вероятность рекомендации.

3 Подход к реализации (FunkSVD)

Поскольку классическое SVD требует плотной матрицы, был реализован подход **FunkSVD** (стохастический градиентный спуск для матричной факторизации).

3.1 Математическая модель

Матрица рейтингов R аппроксимируется произведением двух матриц меньшей размерности P (пользователи) и Q (игры), а также биасами:

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u \quad (1)$$

Где:

- μ — глобальное среднее значение рейтинга.
- b_u, b_i — смещения (biases) пользователя и игры.
- p_u, q_i — латентные векторы пользователя и игры размерности k .

3.2 Обучение

Минимизация функции потерь (RMSE) с регуляризацией L2 для предотвращения переобучения:

$$\min \sum_{(u,i) \in R} (r_{ui} - \hat{r}_{ui})^2 + \lambda(\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2) \quad (2)$$

Оптимизация проводилась методом SGD с параметрами: `lr=0.005`, `reg=0.02`, `epochs=20`.

4 Демонстрация на конкретных примерах

Для лучшего пользователя из обучающей выборки (`pancakes37`) с лучшей конфигурацией модели ($k = 10$) были получены следующие рекомендации:

Таблица 1: Пример рекомендаций для пользователя `pancakes37` (SVD $k=10$)

Rank	Game ID	Game Name	Score
1	312990	The Expendabros	1.0824
2	409160	Dr. Langeskov, The Tiger...	1.0762
3	212800	Super Crate Box	1.0570
4	417860	Emily is Away	1.0398
5	346120	Boring Man - Online...	1.0390

Интерпретация: Предсказанные оценки превышают 1.0, что возможно в моделях матричной факторизации без жесткого ограничения выхода (`clipping` делается только при расчете метрик ошибки). Это указывает на высокую уверенность модели в том, что эти игры понравятся пользователю. Рекомендованные игры относятся к жанрам инди-экшенов и казуальных игр, что, вероятно, соответствует скрытым предпочтениям пользователя, выявленным через латентные факторы.

5 Гиперпараметры модели и влияние их изменения

Основным гиперпараметром SVD является **размерность латентного пространства** (`n_factors` или k). Были протестированы значения: 10, 30, 50, 100.

Таблица 2: Сравнение метрик для разных размерностей латентного пространства

n_factors	RMSE	MAE	Precision@10	Recall@10	HitRate@10	NDCG@10
10	0.3258	0.2019	0.0069	0.0629	0.0695	0.0297
30	0.3264	0.2022	0.0046	0.0418	0.0460	0.0208
50	0.3305	0.2078	0.0035	0.0313	0.0347	0.0125
100	0.3329	0.2053	0.0023	0.0201	0.0234	0.0097

5.1 Сводная таблица результатов

5.2 Интерпретация влияния гиперпараметров

1. Переобучение при росте сложности:

- На тренировочных данных RMSE монотонно уменьшался с ростом k (от 0.2830 при $k = 10$ до 0.2540 при $k = 100$). Это означает, что модель с большим числом факторов лучше “запоминает” тренировочные данные.
- Однако на тестовой выборке **лучшие результаты по всем метрикам (RMSE, Precision, HitRate) показала модель с наименьшей размерностью $k = 10$.**
- При увеличении k до 30, 50 и 100 качество на тесте падало. Это классический признак **переобучения**: модель начинает улавливать шум в данных, а не общие паттерны, особенно учитывая малый объем данных на одного пользователя.

2. Сравнение метрик:

- **RMSE/MAE:** Модель с $k = 10$ имеет наименьшую ошибку предсказания (0.3258).
- **Ranking Metrics (Precision/HitRate):** Также максимальны при $k = 10$. Это говорит о том, что простые латентные профили (10 признаков) лучше обобщают вкусы пользователей в этом датасете, чем сложные (100 признаков).

6 Сравнение с Hybrid KNN

Для полноты картины сравним лучшие результаты SVD ($k = 10$) с лучшими результатами Hybrid KNN ($k = 30, \alpha = 0.3$):

Таблица 3: Сравнение лучших моделей SVD и Hybrid KNN

Метрика	Hybrid KNN (Best)	SVD (Best, k=10)	Победитель
Precision@10	0.0178	0.0069	KNN
HitRate@10	0.1781	0.0695	KNN
RMSE	0.3015	0.3258	KNN
NDCG@10	0.0957	0.0297	KNN

Вывод по сравнению: В данном конкретном случае **Hybrid KNN** показал значительно лучшие результаты, чем SVD.

- **Причины:**

1. **Малый объем данных:** SVD требует большого количества взаимодействий для качественного обучения латентных векторов. При среднем числе отзывов < 2 на пользователя, SVD не успевает выучить значимые признаки и переобучается даже на малых k .
2. **Локальность vs Глобальность:** KNN использует локальное сходство (конкретные соседи), что работает надежнее на маленьких данных. SVD пытается найти глобальную структуру, которая в условиях сильной разреженности оказывается шумной.
3. **Бинарные данные:** SVD лучше работает с непрерывными оценками (1-5). На бинарных данных (0/1) градиентный спуск может вести себя менее стабильно без специальной функции потерь (например, Logistic Loss вместо Square Loss).

7 Выводы

1. **Оптимальная конфигурация SVD:** Для данного датасета наилучшая размерность латентного пространства — **10 факторов**. Увеличение сложности модели приводит к деградации качества на тестовой выборке.
2. **Эффективность на разреженных данных:** Несмотря на теоретические преимущества SVD в обобщении, на крайне разреженных данных с короткой историей пользователей классические методы KNN (особенно гибридные) показывают себя устойчивее и точнее.
3. **Перспективы улучшения SVD:**
 - Использование **Logistic Matrix Factorization** (так как целевая переменная бинарная).
 - Добавление регуляризации или ранняя остановка (Early Stopping) по валидационной выборке.
 - Объединение с контентными признаками (Hybrid SVD), чтобы компенсировать нехватку взаимодействий.
4. **Итоговая рекомендация:** Для текущей задачи рекомендуется использовать **Hybrid KNN** как основную модель. SVD может быть использован как дополнительная модель в ансамбле или для задач, где требуется интерпретация латентных признаков (если данных станет больше).